**TECNOLÓGICO DE MONTERREY**
**MAESTRÍA EN INTELIGENCIA ARTIFICIAL APLICADA**

TC5061 — Operaciones de Inteligencia Artificial (MLOps)
Entrega de Semana 8: Despliegue de Modelos de ML (FastAPI + Contenedores Docker / Podman)

# Reporte Técnico y Ejecutivo: Empaquetado y Despliegue Contenerizado del Microservicio MiBici

| | |
|---|---|
| **Proyecto / Sistema:** | Plataforma de Inteligencia Operativa MiBici (Área Metropolitana de Guadalajara) |
| **Repositorio Oficial GitHub:** | https://github.com/Diegob059/TC5061-Proyecto-Equipo30 |
| **Microservicio Desplegado:** | FastAPI 0.162.0 + Uvicorn 0.54.0 \| Modelo: mibici-equipo30-p2-riesgo@champion |
| **Fecha de Entrega:** | Noviembre de 2026 — Equipo 30 |

## 1. Integrantes del Equipo (Equipo 30)

| Nombre del Integrante | Matrícula |
|---|---|
| Ricardo Bahamón | A01840023 |
| Diego Becerril | A01748115 |
| Gregorio Escobedo Straffon | A01840612 |
| Freddin Alcántara | A01797741 |

En esta actividad integradora, el Equipo 30 da el primer paso hacia la producción industrial del proyecto MiBici mediante el empaquetado de su modelo de predicción de riesgo operativo (Pregunta 2) como un servicio REST interactivo, portable y reproducible en cualquier entorno mediante contenedores OCI

(Docker y Podman). El microservicio expone el clasificador entrenado sobre más de 7.89 millones de viajes históricos, permitiendo al Centro de Control Operativo (CCO) de MiBici anticipar condiciones de desabasto (vaciado) o saturación (llenado) en las 470 estaciones activas con un horizonte táctico de 2 horas.

## 2. Recuperación del Modelo desde el Model Registry (MLflow)

La integración del modelo en la API se realizó bajo un patrón desacoplado y gobernado (api/model_loader.py):

- **Carga por Alias 'champion':** Se recupera dinámicamente 'models:/mibici-equipo30-p2-riesgo@champion', resolviendo a la versión lgbm_profundo (Run ID: 4dcbff8784a348a19f087a0697896be7), seleccionada por su óptimo balance F1 macro y recall en clases minoritarias.
- **Configuración desacoplada (MLFLOW_TRACKING_URI):** Dentro del contenedor, la ubicación del Model Registry se inyecta mediante la variable de entorno MLFLOW_TRACKING_URI (apuntando a http://host.containers.internal:5000 en Podman o http://host.docker.internal:5001 en Docker bajo macOS para evitar colisión con AirPlay). Esta decisión desacopla el ciclo de vida del software del ciclo de vida del modelo.
- **Ventajas operativas:** La imagen OCI permanece inmutable y ligera (~350 MB). Si se promueve un nuevo modelo campeón en MLflow, no se requiere reconstruir la imagen ni reiniciar la infraestructura (Rollback y promoción sin downtime).

## 4. Construcción de la API REST con FastAPI y Validación Pydantic v2

La API implementada en api/main.py está construida con FastAPI bajo el estándar OpenAPI 3.1 y ASGI Uvicorn:

| Método | Endpoint | Propósito y Funcionalidad |
|---|---|---|
| GET | / | Comprobación básica de disponibilidad y nombre del modelo registrado. |
| GET | /health | Health check operativo para sondas de orquestadores y balanceadores (HTTP 200). |
| GET | /model-info | Metadatos detallados de linaje, alias, horizonte temporal (2h) y tracking URI. |
| POST | /predict | Inferencia de clasificación de riesgo en tiempo real (0: Normal, 1: |

| | | |
|---|---|---|
| | | Desabasto, 2: Saturación). |
| GET | /docs | Documentación interactiva Swagger UI con ejemplos reales de Guadalajara. |

Validación con Pydantic v2 (PredictionInput):

- **Restricciones físicas:** capacity > 0 (anclajes físicos), latitud [-90, 90], longitud [-180, 180], flujos históricos >= 0, senos y cosenos cíclicos [-1, 1], y allow_inf_nan=False.
- **Protección contra inyección (extra='forbid'):** Cualquier carga útil que contenga campos ajenos a las 17 variables predictoras del modelo es rechazada automáticamente con código HTTP 422 Unprocessable Entity.
- **Resolución de Incompatibilidad de Esquema MLflow (Bugfix Crítico):** Se implementó una conversión explícita de tipos antes de llamar a model.predict(X): capacity se castea a int32 y las características continuas a float32, resolviendo la incompatibilidad estricta de schema que provocaba errores HTTP 500 al recibir float64.

## 5. Contenerización OCI con Docker y Podman (Dockerfile / Containerfile)

Se implementaron Dockerfile y Containerfile sincronizados bajo el estándar Open Container Initiative (OCI):

- **Imagen Base:** python:3.12-slim, minimizando vulnerabilidades y reduciendo el tamaño a ~350 MB.
- **Soporte OpenMP para LightGBM:** Instalación de libgomp1 mediante apt-get, requisito nativo indispensable para ejecutar el runtime de LightGBM en Linux.
- **Aislamiento de Dependencias:** requirements-api.txt incluye únicamente las 10 dependencias de inferencia (FastAPI, Uvicorn, Pydantic, MLflow, LightGBM, scikit-learn, pandas, numpy, pyasn1 y python-dotenv).
- **HEALTHCHECK Integrado:** Sondeo automático cada 30 segundos sobre http://localhost:8000/health para autoreparación en orquestadores.
- **Automatización en Makefile:** Comandos 'make build_container', 'make run_container', 'make test_api_live' y 'make stop_container' con detección automática del motor disponible (docker o podman).

## 6. Resultados de Verificación, Pruebas Unitarias y Evidencia

El microservicio fue sometido a validación rigurosa mediante dos mecanismos complementarios:

- **Suite de Pruebas Unitarias (pytest):** 62 pruebas pasando al 100% en 13.04 segundos, incluyendo las 8 pruebas de validación de endpoints y esquema de tests/test_api.py.

- **Sondeo Automatizado en Vivo (scripts/verify_container_api.py):** 6 comprobaciones automáticas contra el servicio activo (reports/container_verification.json). La latencia media de inferencia fue de 5.02 ms.




*Figura 1: Evidencia de verificación del contenedor OCI y respuesta en vivo de la API REST.*

## 7. Justificación Teórica y Estratégica del Despliegue

Con base en la literatura fundamental de MLOps (Chip Huyen, 2022, Cap. 7; Treveil et al., 2020, Cap. 6), se justifican las siguientes decisiones de diseño arquitectónico:

- **Inferencia en Tiempo Real (Online) para la Pregunta 2:** La movilidad de bicicletas compartidas en Guadalajara sufre fluctuaciones estocásticas imprevistas (lluvia, eventos masivos, cierres viales). Los despachadores del CCO necesitan evaluar el riesgo inminente de una estación bajo demanda para desviar camiones de rebalanceo en caliente. La API REST proporciona respuestas sub-segundo (< 10 ms) directamente integrables con tableros operativos.
- **Arquitectura Híbrida (Complementariedad con Batch):** Mientras que la Pregunta 2 opera en tiempo real, las Preguntas 1 (cálculo de inventario inicial de bicicletas) y 3 (segmentación de estaciones) se ejecutan como trabajos por lotes (Batch) programados a las 04:00 AM. Este enfoque híbrido evita sobrecostos de infraestructura en la nube y asigna el patrón óptimo a cada necesidad de negocio.

## 8. Trabajo Colaborativo en Git y Gestión de Pull Requests

Siguiendo la política de GitHub Flow y Conventional Commits del equipo, la entrega se organizó en ramas temáticas revisadas y fusionadas mediante Pull Requests formales en el repositorio oficial:

- **PR #34:** fix(api): robustecer validación de esquema MLflow y enriquecer endpoints de salud y metadatos (Autor: Ricardo Bahamón).
- **PR #35:** feat(container): estandarizar Dockerfile OCI y automatizar ciclo de vida en Makefile (Autor: Ricardo Bahamón).
- **PR #36:** docs(readme): guía de despliegue contenerizado y justificación de inferencia en tiempo real (Autor: Ricardo Bahamón).
- **PR #37:** test(container): añadir verificación automatizada y evidencia de ejecución de la API (Autor: Ricardo Bahamón).
- **PR #38:** fix(api): incluir python-dotenv en dependencias del contenedor y blindar importación (Autor: Ricardo Bahamón).

## 9. Conclusiones y Próximos Pasos

La Semana 8 concluye la transformación del modelo de machine learning en un servicio consumible, empaquetado en un contenedor estándar OCI que garantiza reproducibilidad entre entornos de desarrollo y producción.